

Your Code

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <climits>
5 #include <algorithm>
6
7 using namespace std;
8
9 const int ROWS = 5;
10 const int COLS = 5;
11
12 struct Node {
13     int distance;
14     int x;
15     int y;
16
17     bool operator>(const Node& other) const {
18         return distance > other.distance;
19     }
20 };
21
22 void dijkstra(vector<vector<int>>& grid,
23             int startX, int startY,
24             int endX, int endY) {
25
26     vector<vector<int>> dist(
27         ROWS, vector<int>(COLS, INT_MAX)
```

Input

stdin input

Output

```
Shortest Safe Distance: 8
Optimal Path:
(0,0) -> (1,0) -> (2,0) -> (2,1) -
-> (3,1) -> (4,1) -> (4,2) -> (4,3)
-> (4,4)
```

```
28     );
29
30     vector<vector<pair<int, int>>> parent(
31         ROWS,
32         vector<pair<int, int>>(COLS, {-1, -1})
33     );
34
35     priority_queue<
36         Node,
37         vector<Node>,
38         greater<Node>
39     > pq;
40
41     dist[startX][startY] = 0;
42
43     pq.push({0, startX, startY});
44
45     int dx[] = {-1, 1, 0, 0};
46     int dy[] = {0, 0, -1, 1};
47
48     while (!pq.empty()) {
49
50         Node current = pq.top();
51         pq.pop();
52
53         int d = current.distance;
54         int x = current.x;
55         int y = current.y;
56
```

```
56
57     if (d != dist[x][y])
58         continue;
59
60     if (x == endX && y == endY)
61         break;
62
63     for (int k = 0; k < 4; k++) {
64
65         int nx = x + dx[k];
66         int ny = y + dy[k];
67
68         if (nx < 0 || nx >= ROWS ||
69             ny < 0 || ny >= COLS)
70             continue;
71
72         // -1 represents an obstacle
73         if (grid[nx][ny] == -1)
74             continue;
75
76         int newDistance =
77             d + grid[nx][ny];
78
79         if (newDistance < dist[nx][ny]) {
80
81             dist[nx][ny] = newDistance;
82
83             parent[nx][ny] = {x, y};
84
```

```

85         pq.push({
86             newDistance,
87             nx,
88             ny
89         });
90     }
91 }
92 }
93
94 if (dist[endX][endY] == INT_MAX) {
95     cout << "No safe path available.\n";
96     return;
97 }
98
99 cout << "Shortest Safe Distance: "
100     << dist[endX][endY] << endl;
101
102 vector<pair<int, int>> path;
103
104 int x = endX;
105 int y = endY;
106
107 while (x != -1 && y != -1) {
108
109     path.push_back({x, y});
110
111     auto p = parent[x][y];
112
113     x = p.first;

```

```

113         x = p.first;
114         y = p.second;
115     }
116
117     reverse(path.begin(), path.end());
118
119     cout << "Optimal Path:\n";
120
121     for (int i = 0; i < path.size(); i++) {
122
123         cout << "("
124             << path[i].first
125             << ", "
126             << path[i].second
127             << ") ";
128
129         if (i != path.size() - 1)
130             cout << " -> ";
131     }
132
133     cout << endl;
134 }
135
136 int main() {
137
138     /*
139         0   = normal road
140        -1   = obstacle
141         1   = movement cost

```

```

142     */
143
144     vector<vector<int>> grid = {
145         {1, 1, 1, 1, 1},
146         {1, -1, -1, 1, 1},
147         {1, 1, 1, 1, -1},
148         {-1, 1, -1, 1, 1},
149         {1, 1, 1, 1, 1}
150     };
151
152     int startX = 0;
153     int startY = 0;
154
155     int endX = 4;
156     int endY = 4;
157
158     dijkstra(
159         grid,
160         startX, startY,
161         endX, endY
162     );
163
164     return 0;
165 }

```

✓ Submitted